

GIT OCH GITHUB

STEG-FÖR-STEG GUIDE

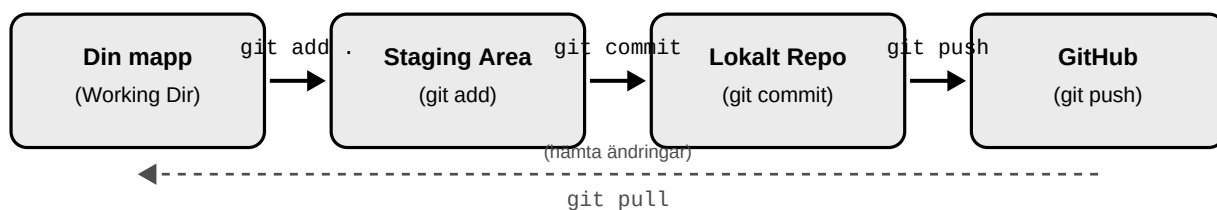
Programmering 1 – JENSEN

Lärare: George Glor

Hur Git fungerar – Översikt

Innan vi börjar med stegen är det bra att förstå hur Git fungerar. Git har fyra "platser" där din kod kan befinna sig:

- **Din mapp (Working Directory)** – Filerna du redigerar i VS Code
- **Staging Area** – Filer som är markerade att sparas (efter git add)
- **Lokalt Repo** – Sparade versioner på din dator (efter git commit)
- **GitHub** – Din kod online, synlig för läraren (efter git push)



Varje gång du vill spara och ladda upp ändringar följer du alltid samma tre steg: **add** → **commit** → **push**. Tänk på det som att packa en låda (add), skriva en etikett (commit) och skicka paketet (push).

Steg 1 – Skapa ett GitHub-konto

1. Gå till www.github.com
2. Klicka på **"Sign up"** (Registrera dig)
3. Fyll i:
 - Din e-postadress (använd din skolmail)
 - Ett lösenord
 - Ett användarnamn (välj något professionellt)
4. Verifiera din e-post via länken du får i mailen

Du är nu redo att använda GitHub!

Steg 2 – Ladda ner och installera Git

1. Gå till git-scm.com/downloads
2. Välj ditt operativsystem (Windows, Mac eller Linux)
3. Ladda ner installationsfilen
4. Öppna filen och följ installationen:
 - Välj alla standardalternativ
 - Klicka "Next" hela vägen
5. När installationen är klar, starta om VS Code

Steg 3 – Öppna terminal i VS Code

-
1. Öppna VS Code
 2. Öppna den mapp där ditt projekt finns:
 - Klicka på "File" → "Open Folder"
 - Välj din projektmapp
 3. Öppna terminalen:
 - Klicka på "Terminal" i toppmenyn
 - Välj "New Terminal"

Eller använd kortkommandot:

- Windows: **Ctrl + `**
- Mac: **Cmd + `**

Steg 4 – Kontrollera att Git är installerat

Skriv detta i terminalen:

```
git --version
```

Du bör se något liknande:

```
git version 2.40.0
```

Om du ser en version är Git installerat!

Steg 5 – Berätta för Git vem du är

Det här gör du bara en gång. Skriv i terminalen:

```
git config --global user.name "Ditt Namn"
```

```
git config --global user.email "din@email.com"
```

OBS: Använd samma e-post som du registrerade på GitHub!

Steg 6 – Skapa ett nytt repository på GitHub

1. Logga in på GitHub
2. Klicka på "+" uppe till höger
3. Välj "New repository"
4. Fyll i:
 - Repository name: **programmering1-projekt**
 - Description: (valfritt)
 - Välj "Public" (så att läraren kan se)

5. KRYSSA INTE i "Add a README file"
6. KRYSSA INTE i "Add .gitignore"
7. KRYSSA INTE i "Choose a license"
8. Klicka på "Create repository"

Steg 7 – Koppla ihop din dator med GitHub

Nu ska du koppla ditt lokala projekt på datorn till GitHub.

7.1 – Initiera Git i din mapp

I terminalen, skriv:

```
git init
```

7.2 – Lägg till alla dina filer

```
git add .
```

7.3 – Spara en första version (commit)

```
git commit -m "Första commit - mitt projekt startar"
```

7.4 – Koppla till GitHub

Gå tillbaka till GitHub-sidan. Där ser du en ruta med text som liknar:

```
git remote add origin https://github.com/ditt-användarnamn/programmering1-projekt.git
```

Kopiera den raden och klistra in den i terminalen.

Exempel:

```
git remote add origin https://github.com/annasvensson/programmering1-projekt.git
```

7.5 – Skicka upp din kod till GitHub

```
git branch -M main
```

```
git push -u origin main
```

Steg 8 – Logga in på GitHub från terminalen

När du kör git push kan ett fönster dyka upp där du ska logga in.

Alternativ 1 – Webbläsare (enklast)

1. Ett fönster öppnas i din webbläsare

2. Logga in på GitHub
3. Klicka på **"Authorize"** (Godkänn)

Alternativ 2 – Personlig åtkomst-token (om webbläsaren inte fungerar)

1. Gå till GitHub → klicka på din profilbild uppe till höger
2. Välj **"Settings"**
3. Scrolla ner och klicka på **"Developer settings"** (längst ner till vänster)
4. Klicka på **"Personal access tokens"** → **"Tokens (classic)"**
5. Klicka på **"Generate new token"** → **"Generate new token (classic)"**
6. Fyll i:
 - Note: "VS Code terminal"
 - Expiration: Välj "90 days"
 - Markera i rutan **"repo"** (alla rutor under repo)
7. Scrolla ner och klicka på **"Generate token"**
8. **KOPIERA TOKEN DIREKT** (du ser den bara en gång!)
9. När terminalen frågar efter lösenord, klistra in token istället för lösenord

Steg 9 – Kontrollera att allt fungerade

1. Gå till GitHub i din webbläsare
2. Öppna ditt repository
3. Du bör nu se dina filer där!

Steg 10 – Vanliga kommandon att använda

När du jobbar med ditt projekt, använd dessa kommandon i terminalen:

Lägg till ändringar

```
git add .
```

Spara en version

```
git commit -m "Beskriv vad du har ändrat"
```

Skicka upp till GitHub

```
git push
```

Hämta senaste versionen från GitHub

```
git pull
```

Se status på dina filer

```
git status
```

Steg 11 – Klona ett projekt från GitHub (git clone)

Om du vill ladda ner ett projekt från GitHub till din dator (t.ex. en klasskamrats projekt eller ett startprojekt från läraren), använd **git clone**.

1. Gå till repositoryt på GitHub
2. Klicka på den gröna knappen "**Code**"
3. Kopiera URL:en (HTTPS)
4. Öppna terminalen i VS Code och skriv:

```
git clone https://github.com/användarnamn/projektnamn.git
```

Git skapar en ny mapp med hela projektet. Öppna den mappen i VS Code för att börja jobba.

Steg 12 – Se din historik (git log)

För att se alla sparade versioner (commits) i ditt projekt, skriv:

```
git log --oneline
```

Du ser något liknande:

```
a3f2d1c Lade till kalkylator  
7b8e9f0 Fixade bugg i loop  
1a2b3c4 Första commit - mitt projekt startar
```

Varje rad är en sparad version. Det korta ID:t (t.ex. a3f2d1c) identifierar den versionen. Tryck **q** för att avsluta listan.

Steg 13 – Ignorera filer med .gitignore

Ibland vill du **inte** ladda upp vissa filer till GitHub – t.ex. temporära filer, lösenord eller saker som Python skapar automatiskt. Då använder du en fil som heter **.gitignore**.

1. Skapa en fil i ditt projekts rotmapp som heter **.gitignore** (exakt så)
2. Skriv in vad som ska ignoreras:

```
# Python-filer som skapas automatiskt
__pycache__/
*.pyc

# Virtuellt miljö
venv/

# Systemfiler
.DS_Store
Thumbs.db

# Hemliga filer (lösenord, nycklar)
.env
secrets.txt
```

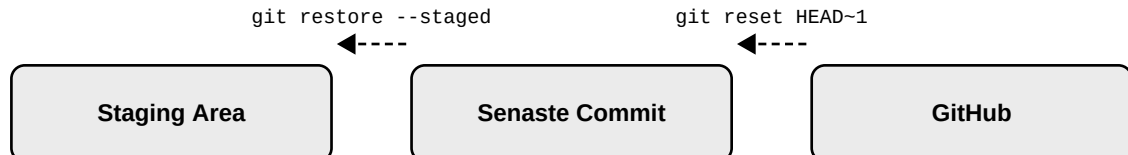
3. Spara filen och kör:

```
git add .gitignore
git commit -m "Lade till .gitignore"
git push
```

Tips: Lägg till .gitignore *innan* du gör din första commit, så laddas skräpfiler aldrig upp.

Steg 14 – När något går fel – ångra ändringar

Alla gör misstag! Här är hur du ångrar saker i Git:



Pilar visar riktningen för ångra-kommandon – de "backar" ett steg.

Ångra ändringar i en fil (innan git add)

Om du har ändrat en fil men vill gå tillbaka till hur den såg ut vid senaste commit:

```
git restore filnamn.py
```

Ta bort en fil från staging (efter git add, innan commit)

Om du körde git add men ångrade dig:

```
git restore --staged filnamn.py
```

Ångra senaste commit (behåll filerna)

Om du körde git commit men vill göra om:

```
git reset HEAD~1
```

Filerna ändras inte – de går bara tillbaka till "ändrade men inte committade".

Ångra allt och börja om från GitHub

Om allt känns fel och du vill börja om från det som ligger på GitHub:

```
git fetch origin
git reset --hard origin/main
```

WARNING: Detta tar bort alla lokala ändringar som inte är pushade!

Vanliga fel och lösningar

Problem	Lösning
git: command not found	Git är inte installerat. Gå tillbaka till Steg 2.
fatal: not a git repository	Du är inte i rätt mapp. Kör git init först.
Permission denied	Du är inte inloggad. Kör git push igen och logga in.
Please tell me who you are	Du har inte kört git config i Steg 5.
failed to push	Någon annan har ändrat. Kör git pull först.
Updates were rejected	Kör git pull --rebase och sedan git push.

Övning – Skicka upp ett enkelt program

1. Skapa en fil som heter **test.py**
2. Skriv in:

```
print("Git fungerar!")
```

3. Spara filen
4. Kör i terminalen:

```
git add .
git commit -m "Lade till test.py"
git push
```

5. Kontrollera på GitHub att filen finns där

Klart!

Du har nu:

- Skapat ett GitHub-konto
- Installerat Git
- Kopplat ihop VS Code med GitHub

- Skickat upp din första kod
- Lärt dig klona, se historik, ignorera filer och ångra misstag

Nu kan du använda GitHub för att spara alla dina projekt!

GIT CHEAT SHEET – Snabbreferens

Skriv ut den här sidan och ha bredvid tangentbordet!

Starta

<code>git init</code>	Starta Git i en mapp
<code>git clone URL</code>	Ladda ner ett projekt från GitHub

Dagligt arbete

<code>git status</code>	Se vilka filer som är ändrade
<code>git add .</code>	Markera alla ändringar för sparning
<code>git add filnamn</code>	Markera en specifik fil
<code>git commit -m "meddelande"</code>	Spara en version
<code>git push</code>	Skicka till GitHub
<code>git pull</code>	Hämta från GitHub

Se historik

<code>git log --oneline</code>	Se alla sparade versioner (kort)
<code>git log</code>	Se detaljer om varje version
<code>git diff</code>	Se vad som ändrats

Ångra

<code>git restore filnamn</code>	Ångra ändringar i en fil
<code>git restore --staged filnamn</code>	Ta bort fil från staging
<code>git reset HEAD~1</code>	Ångra senaste commit (behåll filer)
<code>git reset --hard origin/main</code>	Börja om från GitHub (VARNING!)

Konfiguration

<code>git config --global user.name "Namn"</code>	Sätt ditt namn
<code>git config --global user.email "e@mail"</code>	Sätt din e-post
<code>git remote -v</code>	Se vilken GitHub-URL som är kopplad

Kom ihåg: *add → commit → push. Varje gång!*